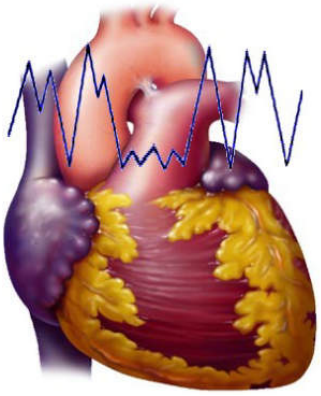
Équations différentielles
MODÉLISATION ET SIMULATION DU BATTEMENT DU CŒUR

1 Description et mise en situation

Le muscle cardiaque, comme tous les muscles du corps, se contracte sous l'influence d'une impulsion électrique qui le traverse périodiquement. Cette impulsion se transmet à travers le myocarde par l'intermédiaire de structures tissulaires spécialisées (les voies de conduction anatomiques), mais se produit également de cardiomyocyte à cardiomyocyte par un mécanisme de propagation par contiguïté. Les cellules myocardiques sont à chaque instant chargées électriquement et se comportent comme des dipôles électriques, variant entre une charge électrique positive ou négative, en fonction du potentiel d'action et de leur statut dépolarisé (potentiel positif) ou repolarisé (potentiel négatif) (Voir la figure ci-dessous).



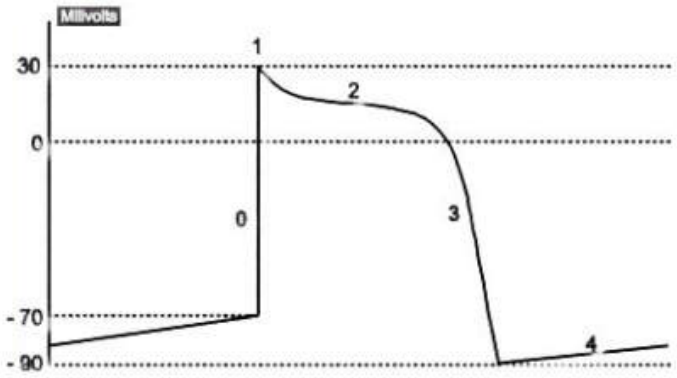


FIGURE 1 – Les différentes phases du potentiel d'action cardiomyocytaire

Le battement du cœur fait partie des systèmes naturels auto-excités (à partir de n'importe quelle condition initiale, le système approche rapidement un cycle limite stable) pour lesquels on peut remarquer des oscillations avec relaxation. En citant Van der Pol et Van der Mark, plusieurs phénomènes présentent ce type d'oscillations : harpe éolienne, marteau pneumatique, grincement d'un couteau sur une assiette, mouvement d'un drapeau dans le vent,... tube à néon,... et, finalement, le battement du cœur.

Ces phénomènes se caractérisent par les propriétés suivantes :

- Leur période est constante (temps de relaxation) ;
- La forme de l'onde est sensiblement différente d'une onde sinusoïdale ;
- L'amplitude de l'onde est indépendante de la force extérieure appliquée pourvu que cette force soit assez petite ;
- La période, en revanche, dépend de la force extérieure appliquée. Si celle-ci est périodique, le système de relaxation tend à se synchroniser pour devenir périodique avec la même période que celle de la force extérieure ...

L'équation de Van der Pol servant à modéliser les battements du cœur et qui régit le comportement de certains oscillateurs à un degré de liberté est du type :

$$\frac{d^2x(t)}{dt^2} - \mu\left(1 - x^2(t)\right)\frac{dx(t)}{dt} + x(t) = 0$$

L'étude de cette équation montre qu'il y a en général convergence vers un régime périodique indépendant des conditions initiales.

On note pour la suite :

$$v(t) = \dot{x}(t) = \frac{dx(t)}{dt}$$

2 Résolution avec deux méthodes d'intégration

Activité 1

- Exprimer la dérivée de $v(t)$ en fonction de v, de x et des paramètres régissant l'équation différentielle de Van der Pol.

Après discrétisation et pour pouvoir intégrer l'équation différentielle précédente, on note x_k et v_k les valeurs approchées de $x(t_k)$ et de $v(t_k)$. t_k étant obtenu à chaque pas de temps de calcul noté h.

Activité 2

- Exprimer alors v_{k+1}, x_{k+1}, et t_{k+1} en fonction de t_k, x_k, v_k et h.

Activité 3

- Concevoir une fonction `F` qui permet de calculer v_{k+1}.

```
import numpy as np
from scipy.integrate import odeint
from matplotlib import pyplot as plt

def F(x,v,t,mu):
    a = mu*(1-x**2)*v - x
    return a
```

On note pour la question suivante les arguments suivants :

- `F` : la fonction définie précédemment ;
- `a` : la date initiale pour la résolution ;
- `b` : la date finale pour la résolution ;
- `xinit` : la condition initiale sur x(t) ;
- `vinit` : la condition initiale sur v(t) ;
- `h` : le pas de de temps de calcul.

Activité 4

- Programmer une fonction `euler_vdp(F,a,b,xinit,vinit,h,mu)` qui permette de résoudre l'équation différentielle de Van der Pol. Cette fonction doit retourner trois listes, une liste des temps, une liste des positions et une liste des vitesses obtenues à chaque pas de calcul.

```
def euler_vdp(phi,a,b,xinit,vinit,h,mu):
    liste_t=[a];
    liste_v=[vinit];
    liste_x=[xinit];
    t=a; v=vinit; x=xinit;
    while t+h<=b:
        v = v + phi(x,v,t,mu) * h
        x = x + v * h
        t = t + h
        liste_t = liste_t + [t]
        liste_v = liste_v + [v]
        liste_x = liste_x + [x]
    return (liste_t,liste_x,liste_v)
```

On commence tout d'abord par fixer les valeurs : $\mu = 1$; $x(0) = 0$ et $v(0) = 0,01$.

Activité 5

- Résoudre cette équation différentielle et tracer sur une même figure l'évolution de x et de v en fonction du temps sur l'intervalle $[0\,;\,30]$ pour 3 pas de discrétisation différents : 0,1, 0,01 et 0,005. On utilisera la fonction `subplot` du module `matplotlib.pyplot` pour représenter x et v, l'un en dessous de l'autre.

```
liste_h=[0.1,0.01,0.005]

plt.figure()
```

```
for h in liste_h:
    T0,X0,V0 = euler_vdp(F,0,30,0,0.01,h,1)
    plt.subplot(2,1,1)
    plt.plot(T0,X0,label="h = {}".format(h))
    plt.xlabel("t"); plt.ylabel("x")
    plt.legend(loc='best',fontsize=10)
    plt.subplot(2,1,2)
    plt.plot(T0,V0,label="h = {}".format(h))
    plt.xlabel("t"); plt.ylabel("v")
    plt.legend(loc='best',fontsize=10)
```

Activité 6

- Analyser les résultats. Que peut-on en conclure ?

Nous allons maintenant utiliser la méthode `odeint` de la bibliothèque `integrate` du module `scipy`. L'annexe ci-dessous fournit une version simplifiée de l'aide Python sur la fonction `odeint`.

```
Definition: odeint(func, y0, t, args)
Type: Function of scipy.integrate.odepack module
----
Integrate a system of ordinary differential equations.
Solve a system of ordinary differential equations using lsoda from the FORTRAN library odepack.
Solves the initial value problem for stiff or non-stiff systems of first order ode-s::
dy/dt = func(y,t0,...)
where y can be a vector.
Parameters
----------
func : callable(y, t0, ...)
Computes the derivative of y at t0.
y0 : array
Initial condition on y (can be a vector).
t : array
A sequence of time points for which to solve for y. The initial value point should be the first
    element of this sequence.
args : tuple, optional
Returns
-------
y : array, shape (len(t), len(y0))
Array containing the value of y for each desired time in t, with the initial value 'y0' in the
    first row.
```

Remarque

les arguments doivent être renseignés sous forme de `tuple` :

- Si nous avons deux arguments u et v, on saisira alors, `args=(u,v)` ;
- Dans le cas d'un seul argument u, on saisira `args=(u,)`.

Activité 7

- Résoudre l'équation différentielle de Van der Pol avec cette fonction `odeint` pour les 3 valeurs de pas de discrétisation définies au-dessus ;
- Tracer les mêmes courbes que précédemment.

```
def G(Y,t,mu):
    v = Y[1]
    a = mu*(1-Y[0]**2)*Y[1]-Y[0]
    return (v,a)

def ode_vdp(phi,a,b,xinit,vinit,h,mu):
    N=1+(b-a)//h
    liste_t=np.linspace(a,b-h,N)
    Z=odeint(phi,[xinit,vinit],liste_t,args=(mu,))
    liste_x=[z[0] for z in Z]
    liste_v=[z[1] for z in Z]
    liste_t=liste_t.tolist()
    return (liste_t,liste_x,liste_v)

liste_h=[0.1,0.01,0.005]

plt.figure()
for h in liste_h:
    T,X,V = ode_vdp(G,0,30,0,0.01,h,1)
    plt.subplot(2,1,1)
    plt.plot(T,X,label="h = {}".format(h))
    plt.xlabel("t"); plt.ylabel("x")
    plt.legend(loc='best',fontsize=10)
    plt.subplot(2,1,2)
    plt.plot(T,V,label="h = {}".format(h))
    plt.xlabel("t"); plt.ylabel("v")
    plt.legend(loc='best',fontsize=10)
```

Nous allons comparer les résultats des deux méthodes d'intégration d'équations différentielles en calculant les écarts obtenus par chacune de ces méthodes.

Activité 8

- Imaginer et concevoir un programme permettant de calculer et de tracer pour les 3 pas de discrétisation les écarts obtenus sur x en fonction du temps.

```
# Ecarts entre les méthodes ode et euler :

plt.figure()
liste_h=[0.1,0.01,0.005]
for h in liste_h:
    T0,X0,V0 = euler_vdp(F,0,30,0,0.01,h,1)
    T,X,V = ode_vdp(G,0,30,0,0.01,h,1)
    n = min(len(T),len(T0))
    dX = []
```

```
    for k in range(n):
        dX += [X[k]-X0[k]]
    plt.plot(T[:n],dX,label="h = {}".format(h))
    plt.xlabel("t"); plt.ylabel("ecart x")
    plt.legend(loc='best',fontsize=10)

plt.show()
```

Remarque

Dans la suite, on utilisera exclusivement `odeint` du module `scipy.integrate` et un pas de temps de calcul $h = 0,01$.

3 Étudier la dépendance des conditions initiales

On reprend la condition initiale suivante $x(0) = 0$; et la valeur de $\mu = 1$.

Activité 9

- Tracer sur un même graphe les différentes solutions (en position $x(t)$) correspondant aux conditions initiales supplémentaires suivantes : $v(0) = 0,001$, $v(0) = 0,01$, $v(0) = 0,1$ et $v(0) = 1$ pour $t \in [0, 30]$;
- Que constate-t-on ?

```
# Definition des fonctions utilisées

def G(Y,t,mu):
    v = Y[1]
    a = mu*(1-Y[0]**2)*Y[1]-Y[0]
    return (v,a)

def ode_vdp(phi,a,b,xinit,vinit,h,mu):
    N=1+(b-a)//h
    liste_t=np.linspace(a,b-h,N)
    Z=odeint(phi,[xinit,vinit],liste_t,args=(mu,))
    liste_x=[z[0] for z in Z]
    liste_v=[z[1] for z in Z]
    liste_t=liste_t.tolist()
    return (liste_t,liste_x,liste_v)

# Definition des C.I. :
h=0.01
x0=0; mu=1
liste_v0=[0.001,0.01,0.1,1]
ti=0; tf=30

# Tracé des courbes
for v0 in liste_v0:
    T,X,V = ode_vdp(G,ti,tf,x0,v0,h,mu)
```

```
    plt.figure(1)
    plt.plot(T,X,label="v0 = {}".format(v0))
    plt.xlabel("t"); plt.ylabel("x")
    plt.legend(loc='best',fontsize=10)
    plt.figure(2)
    plt.plot(X,V,label="v0 = {}".format(v0))
    plt.xlabel("x"); plt.ylabel("v")
    plt.legend(loc='best',fontsize=10)
plt.show()
```

4 Étudier la dépendance du paramètre μ

On fixe maintenant les conditions initiales suivantes $x(0) = 2$ et $v(0) = 1$.

On souhaite étudier l'influence du paramètre μ sur l'évolution temporelle de x. Pour ce faire, on prendra toutes les valeurs de μ entre 0 et 4 (valeurs incluses), espacées de 0,5.

Activité 10

- Imaginer et concevoir un programme permettant d'obtenir une figure décrivant l'évolution temporelle de x permettant de superposer les tracés obtenus pour ces différentes valeurs de μ. On fera impérativement un programme itératif avec une boucle `while`.

```
# Definition des fonctions utilisées

def G(Y,t,mu):
    v = Y[1]
    a = mu*(1-Y[0]**2)*Y[1]-Y[0]
    return (v,a)

def ode_vdp(phi,a,b,xinit,vinit,h,mu):
    N=1+(b-a)//h
    liste_t=np.linspace(a,b-h,N)
    Z=odeint(phi,[xinit,vinit],liste_t,args=(mu,))
    liste_x=[z[0] for z in Z]
    liste_v=[z[1] for z in Z]
    liste_t=liste_t.tolist()
    return (liste_t,liste_x,liste_v)

# Definition des C.I. :
h=0.01
x0=2; v0=1
mu=0;
ti=0; tf=30

# Tracé des courbes
while mu<=4:
    T,X,V = ode_vdp(G,ti,tf,x0,v0,h,mu)
    plt.figure(1)
    plt.plot(T,X,label="$\mu$ = {}".format(mu))
    plt.xlabel("t"); plt.ylabel("x")
    plt.legend(loc='best',fontsize=10)
```

```
    plt.figure(2)
    plt.plot(X,V,label="$\mu$ = {}".format(mu))
    plt.xlabel("x"); plt.ylabel("v")
    plt.legend(loc='best',fontsize=10)
    mu+=0.5
```

On peut observer que la période du régime permanent dépend de μ. On admettra que cette période est égale à la différence des abscisses de deux maximums consécutifs.

Activité 11

- Définir un programme calculant la liste des maximums de x pour $t \in [0, 30]$ puis retournant la valeur moyenne de la période ;
- Tracer alors le graphe représentant la valeur de la période en fonction de μ pour des valeurs de μ définies entre 0 et 4 (avec un pas de 0,1).

```
def maxis(X):
    """delivre les maximums locaux et indices correspondants d'une liste"""
    indices_max=[]; val_maxis=[];
    for i in range(1,len(X)-1):
        if X[i-1]<=X[i] and X[i]>X[i+1]:
            val_maxis.append(X[i])
            indices_max.append(i)
    return indices_max, val_maxis

def periode(T,X):
    imax, valmax = maxis(X)
    liste_T = []
    for k in range(len(imax)-1):
        liste_T.append(T[imax[k+1]]-T[imax[k]])
    S = 0
    for k in range(len(liste_T)):
        S += liste_T[k]
    S = S/len(liste_T)
    return S

mu=0
liste_mu = []
liste_T = []
while mu<=4:
    liste_mu.append(mu)
    T,X,V = ode_vdp(G,ti,tf,x0,v0,h,mu)
    liste_T.append(periode(T,X))
    mu+=0.1

plt.figure()
plt.plot(liste_mu,liste_T)
plt.xlabel("$\mu$"); plt.ylabel("$Période$ $T$")
plt.show()
```

5 Oscillations forcées

Lorsque cet oscillateur est excité par un terme harmonique de pulsation ω, l'équation différentielle devient :

$$\frac{d^2x(t)}{dt^2} - \mu\left(1 - x^2(t)\right)\frac{dx(t)}{dt} + x(t) = A\sin(2\pi Ft)$$

Activité 12

- Observer le comportement de cet oscillateur pour des conditions initiales nulles et avec : $\mu = 8,53$; $A = 1,2$ et $F = 0,1$.

```python
import numpy as np
from scipy.integrate import odeint
from matplotlib import pyplot as plt

# Définition des fonctions utilisées

def G2(Y,t,mu,A,w):
    v = Y[1]
    a = mu*(1-Y[0]**2)*Y[1] - Y[0] + A*np.sin(2*np.pi*w*t)
    return (v,a)

def ode_vdp2(phi,a,b,xinit,vinit,h,mu,A,w):
    N=1+(b-a)//h
    liste_t=np.linspace(a,b-h,N)
    Z=odeint(phi,[xinit,vinit],liste_t,args=(mu,A,w))
    liste_x=[z[0] for z in Z]
    liste_v=[z[1] for z in Z]
    liste_t=liste_t.tolist()
    return (liste_t,liste_x,liste_v)

# Resultat :
T,X,V = ode_vdp2(G2,0,120,0,0,0.01,8.53,1.2,0.1)
plt.figure()
plt.plot(T,X)
plt.xlabel("t"); plt.ylabel("x")
plt.show()
```